

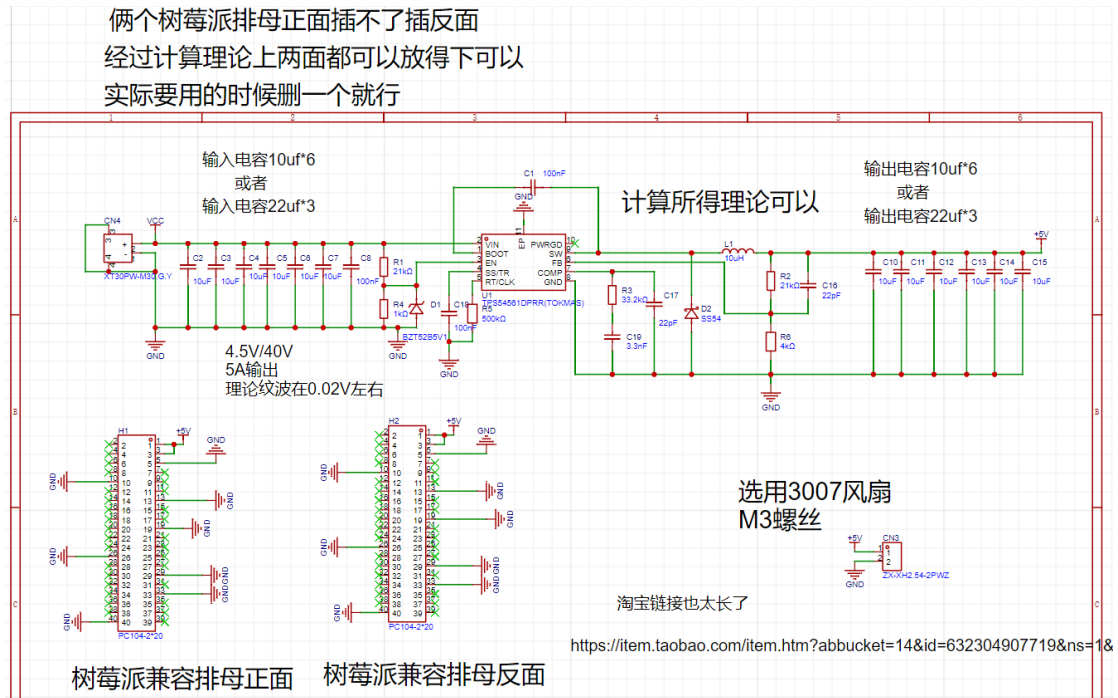
智能车校赛作品

任务报告

北方工业大学 QRS959041

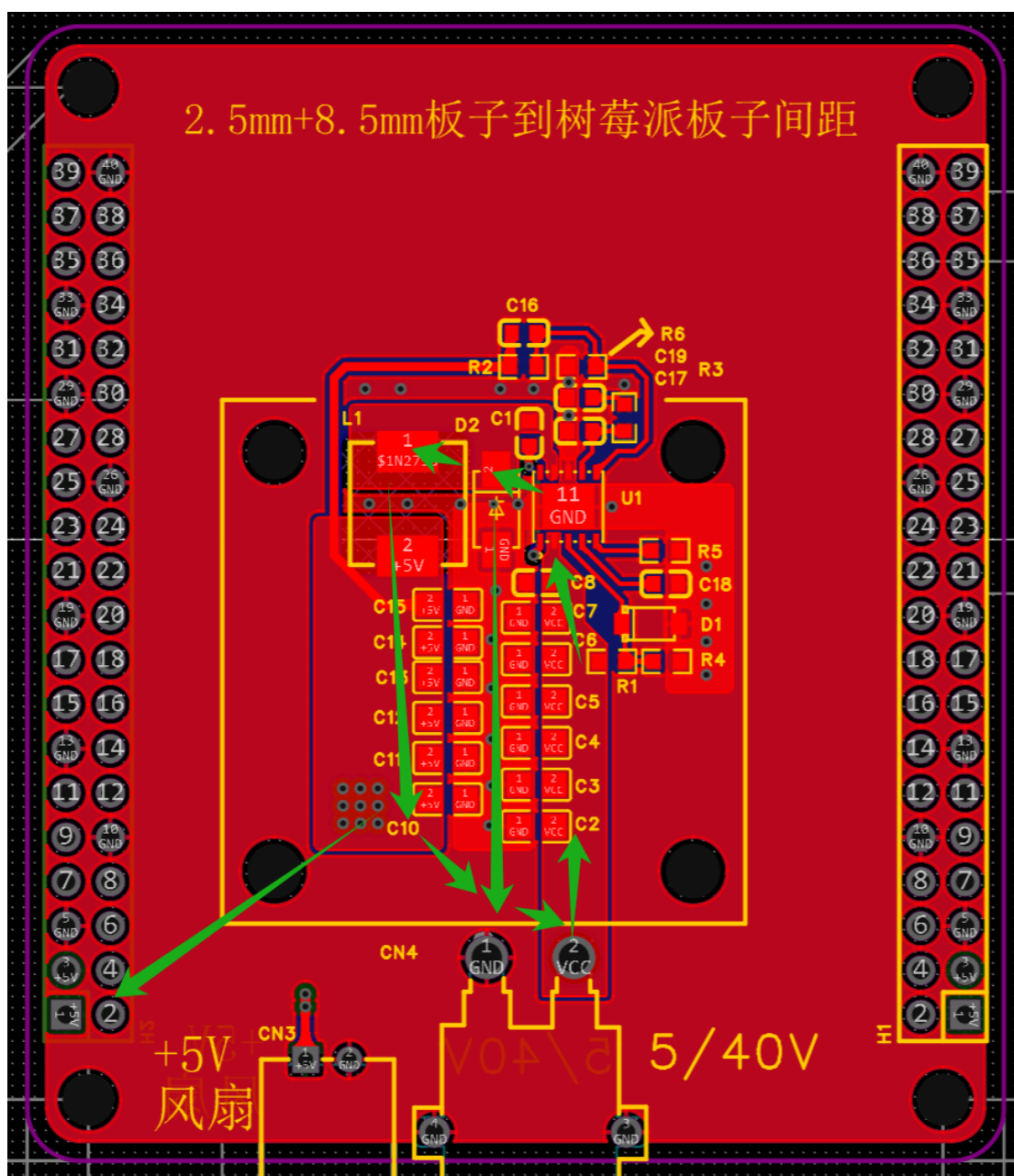
一. 硬件原理图及 PCB 制作

1.1 原理图

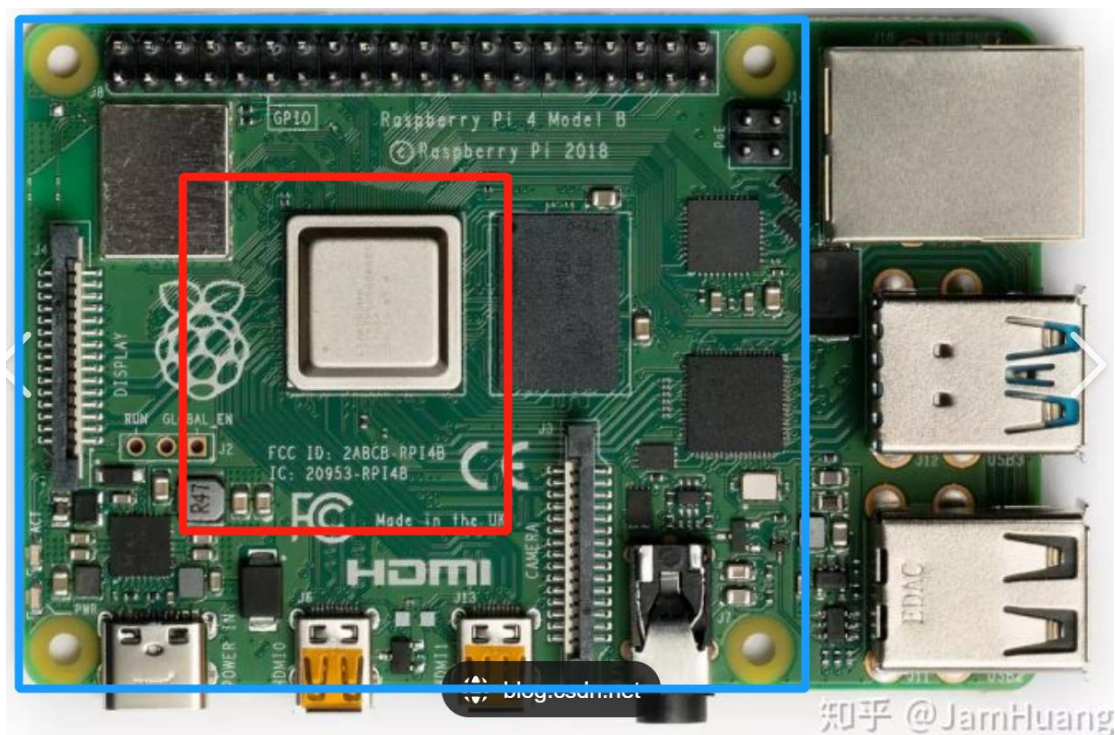


1. 使用 TPS54561 理论可以支持 4.5V~60V 输入，但是我工程里用的电容是 50v 耐压，续流二极管是 40V 耐压，所以输入还是尽量别超过 40V。树莓派要稳定运行至少需要一个 5V/3A 的电源(没有其他外接的设备情况下)，这个芯片最大可支持 5A 电流输出（选 5A 最大输出，主要怕它跑满的情况下温升太大）。
2. 这些个输入/输出电容、电感根据公式算过可以实现，官方给的推荐电感太贵。
3. 我工程里的两个排针到最后只留一个怕正面的安不上就又留了一个反面的（按照测量结果是正好可以装上的）。

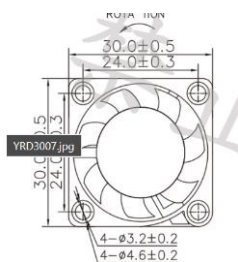
1.2 PCB 板子和安装位置



Buck 电路环路尽量最小，因为这个电路走 3A 的时候散热需求比较小还有风扇散热，所以我在电感下面全部挖空减少电磁干扰（虽然这个板子已经没有可以干扰的东西了）。供电口为 xt30，旁边装一体式的风扇接口 2.54mm



板子大小如蓝色框选部分，红色部分为风扇位置（大致位置）



风扇规格 3007

二. 软件部分

2.1 Shell 程序介绍

(1) Letter Shell 3.x 使用：

自己创建 c 文件里面做如下内容：

- 1.初始化 shell 结构体
- 2.创建缓存区
- 3.里面写好自已的串口接收、发送程序

注意 串口接收程序需要给一个返回值接收到数据返回 1，没接收到超时返回 0；按照这个逻辑可以改成中断/dma 接收这样可以调整 main 函数里的 while 循环的执行速度（多久执行一次）。

- 4 . 创建 shell_init 函数

```

void shell_Init()
{
    ... shell.read = shellRead;
    ... shell.write = shellWrite;
    ... shellInit(&shell, shellBuffer, 512);
}

```

shell.read 指向串口读取函数

shell.write 指向串口发送函数

shellInit (&创建的 shell 结构体, 缓冲区, 缓冲区大小)

5.默认情况在 main 函数 while 循环上面执行:【shellTask 函数内部是一个死循环不用写到 while 里面 (除非你把 SHELL_TASK_WHILE 这个定义成 0)】。

shell_Init();

shellTask(&shell);

```

9
10 //创建一个shell结构体
11 Shell shell;
12 //缓冲区
13 char shellBuffer[512];
14
15 //这里很重要串口等待时间决定了main函数里while循环任务执行情况的灵敏度
16 //太高了任务会比你规定的执行速度慢
17 //比如说你1s内一次灯等待时长设成300你执行的时候可能时1.2秒闪一次
18 //这个可以改成中断或DMA,稍微处理一下就行逻辑挺简单的。这次用不上懒得弄 ("←")
19 short shellRead(char* data,unsigned short len)
20 {
21     ... if (HAL_UART_Receive(&hlpuart1,(uint8_t*)data,len,5) != HAL_OK)
22     {
23         ... return 0;
24     }
25     ... else
26     {
27         ... return 1;
28     }
29 }
30
31 short shellWrite(char* data,unsigned short len)
32 {
33     ... HAL_UART_Transmit(&hlpuart1,(uint8_t*)data,len,0x1FF);
34     ... return len;
35 }
36
37
38
39
40 void shell_Init()
41 {
42     ... shell.read = shellRead;
43     ... shell.write = shellWrite;
44     ... shellInit(&shell, shellBuffer, 512);
45 }
46

```

(2) shell 执行逻辑

```

... struct
... {
    ... unsigned short length; ... /**< 输入数据长度 */
    ... unsigned short cursor; ... /**< 当前光标位置 */
    ... char *buffer; ... /**< 输入缓冲 */
    ... char *param[SHELL_PARAMETER_MAX_NUMBER]; ... /**< 参数 */
    ... unsigned short bufferSize; ... /**< 输入缓冲大小 */
    ... unsigned short paramCount; ... /**< 参数数量 */
    ... int keyValue; ... /**< 输入按键键值 */
    ... } parser;

```

1.Main 函数执行 shellTask(), 进到访问串口读取情况的 while 循环里 (相当于串口相关的中断服务/中断回调函数) 然后调用 shellHandler;

2.进行输入键值过滤, 选取有效字节

3.用户通过串口发送给单片机的内容会先存到 buffer 里面 (创建的缓冲区)

4.先访问任务列表万一有定义好的按键处理命令 (比如说这次任务的 Ctrl+c 或者定义快捷键)。

5. 正常没有按键任务, 进入 shellNormalInput 函数然后跳转 shellInsertByte 函数 (按光标位置插入字符)。

6.有按键任务如回车——执行命令

```

void shellEnter(Shell *shell)
{
    ... shellExec(shell);
    ... shellWritePrompt(shell, 1);
}
#ifdef SHELL_ENTER_LF == 1
SHELL_EXPORT_KEY(SHELL_CMD_PERMISSION(0)|SHELL_CMD_ENABLE_UNCHECKED,
0x0A000000, shellEnter, enter);
#endif

```

7. 进入 shellExec 函数先执行 shellHistoryAdd() 添加历史输入, 然后进入 shellParserParam() 函数以空格为标志拆分参数, 并存到 shell 结构体里 param 数组里面,

8. 把 param 的第一个数组内容传到 shellSeekCommand 里面去找 commandList 里面匹配命令并返回命令地址指针

9. 匹配到的命令跳转到 shellRunCommand 函数执行命令

10. 判断命令类型跳转到对应命令类型处理的 if 语句里面写返回值

//中间还有一些细节, 命令行处理, 参数数据处理和传递, 命令输入提示和反馈, 错误处理。

本次任务里常用的三个宏: (主要用在退格键检测, 回车任务确定检测, Ctrl+c 检测, 任务指令添加)

命令增加宏: 两个

SHELL_EXPORT_CMD(SHELL_CMD_PERMISSION(0)|SHELL_CMD_TYPE(SHELL_TYPE_CMD_MAIN), func, func, test); // 这个是调用的函数不含参数的 (要输入的内容, 要执行的函数, 任务描述)

SHELL_EXPORT_CMD(SHELL_CMD_PERMISSION(0)|SHELL_CMD_TYPE(SHELL_TYPE_CMD_FUNC), func, func, test); // 这个是调用的函数含参数的 (要输入的内容, 要执行的函数, 任务描述)

按键检测宏

SHELL_EXPORT_KEY(SHELL_CMD_PERMISSION(0)|SHELL_CMD_ENABLE_UNCHECKED, 0x03000000, Ctrl_c, Ctrl_c); // (按键值, 调用的函数, 命令描述)

2. 2flash

1. Flash 用 spi 通信的一般步骤

- (1) 初始化 SPI 接口, 配置正确的模式、时钟速度、数据位顺序。
- (2) 使用 CS 引脚选择芯片。
- (3) 发送特定命令 (如读、写、擦除等)。
- (4) 发送地址或数据, 根据命令要求。
- (5) 读取或写入数据。
- (6) 释放 CS 引脚。
- (7) 检查状态寄存器, 等待操作完成 (如写入或擦除时)。

2. 这次主要用的函数:

- (1) BSP_W25Qx_Init()

复位 -> 擦除扇区 -> 覆盖为零 (方便第一次跟 imu 读到的加速度作比较好把数据存进去, 要不默认里面存的是 0xff)。

(2) BSP_W25Qx_WriteEnable()
拉低片选->发送写使能命令->拉高

(3) BSP_W25Qx_Read ()
拉低片选->发送读指令+3 字节的地址->读取指令->拉高

(4) BSP_W25Qx_Write ()
计算地址写循环框架->存地址->写使能->拉低片选->发送写指令+3 字节地址->发送要写的数据+3 字节地址->拉高->判断是否结束或超时 (循环)

2.3imu

1 . BMI160_Read()
通过 iic 协议发送 (iic 地址 0x69+发送要读取的寄存器地址)

2 . BMI160_Write()
写入(要写的地址+要发数据)

3. getAccelerometerValue()
(1) 发送数据让 imu 工作在正常工作模式
(2) 通过中断定时 100ms 当 imu 准备时间
(3) 分别读取 x, y, z 轴上的分加速度 (2 个字节) 然后做数据处理
(4) 计算出总的加速度后若大于 1.01g (自然状态下有 1.006g 的总加速度) 就与 flash 里存的历史记录的最大加速度作比较; 大就更新 flash 里存的数据。
(5) 执行 5 次后就够 500ms 左右了进行一次播报展示板子状态声明一个 broadcast 静态变量。

4. 执行逻辑

IMU_DEMO 进行任务初始化, imu_flag=1; 在主循环的 Task_run 函数里访问 getAccelerometerValue()实现 imu 整体功能

2.4led

因为要用呼吸灯所以直接把 io 口配置成 pwm 信号输出了, 控制占空比来控制小灯亮灭实现闪烁。__HAL_TIM_SetCompare(&htim2,TIM_CHANNEL_1,led_brightness);

(1) ON: led_control(0,1,1);->__HAL_TIM_SetCompare(&htim2,TIM_CHANNEL_1,1000);

(2) OFF: led_control(1,1,1);-> __HAL_TIM_SetCompare(&htim2,TIM_CHANNEL_1,0);

(3) blink: led_control(2,-1,1000);-> 每进一次 TIM7 中断 led_time7_now+1 够 1000ms 后把 led_brightness 改成 0 or 1000。再加一个判断 c 是否为负的语句判断是否一直闪。

(4) breath: led_control(3,-1,1000);-> 添加一个 turn 的标志位, 让 1/20, 当 led_time7_now==1/20 后让 led_brightness +/- (max_led_brightness/20) 再加一个判断 c 是否为负的语句判断是否一直呼吸。

(5) led_ctl: led_control(M,C,I);->其实就是存好"--m"、"--c"、"--i"、"on"、"off"、"blink"、"breath"7 个字符串 (因为就 ledctl 一个指令用也没必要单独存一个字符列表或者哈希表什么的), 创建指针*src= shell.parser.param[count];指向输入的命令行参数数组。因为输入的位置是固定的所以挨个比较, 做好错误输入处理就行了。用 strcmp 函数就行我写的时候不知道有这么个函数懒得改了。

任务逻辑:

有关 led 的命令都调用 led_control 进行任务初始化【无非是参数处理】，led_flag=1；在主循环的 Task_run 函数里访问 led_run()确定 led 的执行情况。

2. 5beep

用 TIM6 定时翻转 io 电平模仿 pwm 信号 440hz 就是说 Prescaler 设置为 170-1，计数 2272 时 io 口要输出一次高电平在输出一次低电平，所以计数 1136 次时翻转一次电平。主要蜂鸣器和 led 灯共用一个定时器 TIM2 所以就这么处理了。其实都用一个定时器也行无非就是呼吸灯占空比的数不好看。按我这个程序来说每次占空比+113？我就喜欢整数那咋办？

任务逻辑：

beep_start()进行任务初始化，beep_flag=1；开启 TIM6 定时中断；在主循环的 Task_run 函数里访问 beep_run()实现 beep 响 0.5s。

三. 未来优化

- 1.把 shell 的串口读/写改成 dma 形式的，shell 经常使用串口可以加快运行速度。
- 2.加几个变量把可以在 cube 里更改的数值存到变量里，替代掉程序里我手算的数字可以方便移植。
- 3.错误处理没做完，iic 或者 spi 超时没有进行处理，没有重新执行什么的，可能导致数据错误或程序中断。
- 4.万一以后要移植的话得把 beep 改成 tim2 的 pwm 信号，软件 pwm 功耗、精度、资源占用都不如硬件 PWM 利用专用的定时器硬件提供的精度、稳定性和效率。
- 5.这个 letter shell 的功能太多兼容 frtos，这个没操作系统的程序很多部分没用上，把没用的部分删了，或者了解一下其他功能看看有没有更好的实现方法。
- 6.把 led_ctl 里面重复的程序改成用 strcmp 简化一下。
- 7.pcb 可以焊一个测试一下确定装正面还是反面，然后把多余的电容和排针删了。